CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 7: Queue Implementation using Array and Linked List, with Structure Simulation

Practical Aim: To implement queue using array and linked list, and apply queue-based logic to number generation, and structure simulation problems.

Problem 1:

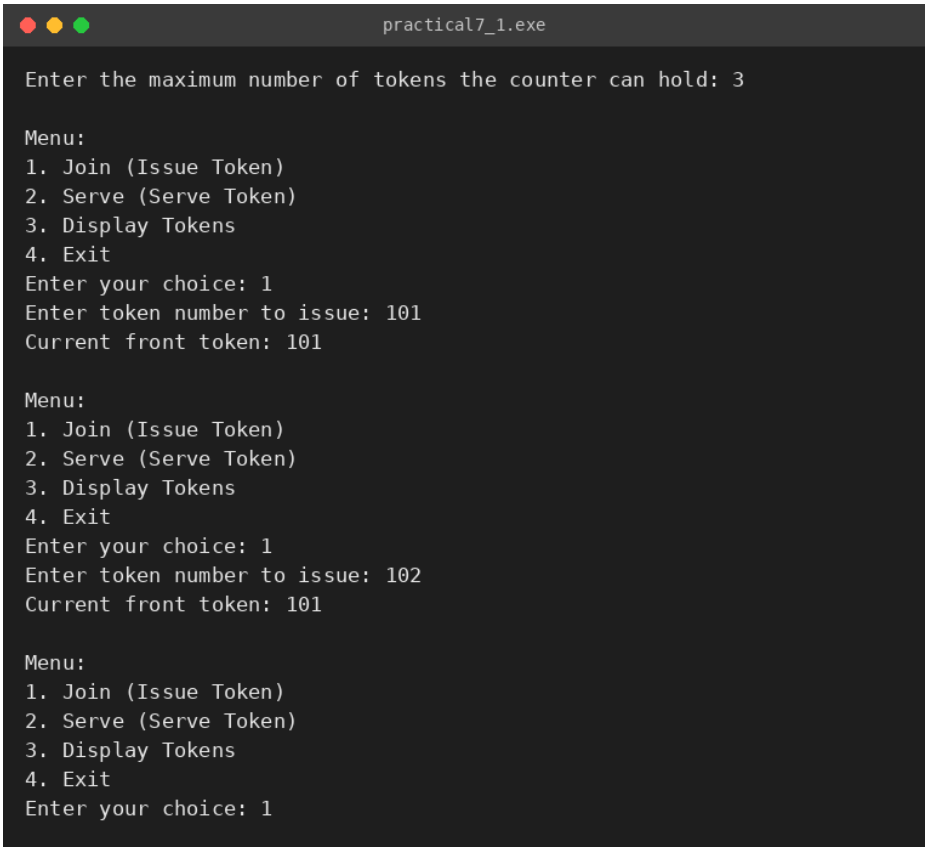
A government office has a token counter that issues tokens to visitors. The counter can hold at most n tokens at a time. New visitors join from one end and are served from the other end in the order they arrived. If the counter is full, no new token can be issued, and if it is empty, no one can be served. Given a sequence of join and serve operations, implement this fixed-capacity token system and print the current front token after each operation. Report an error if an operation cannot be performed.

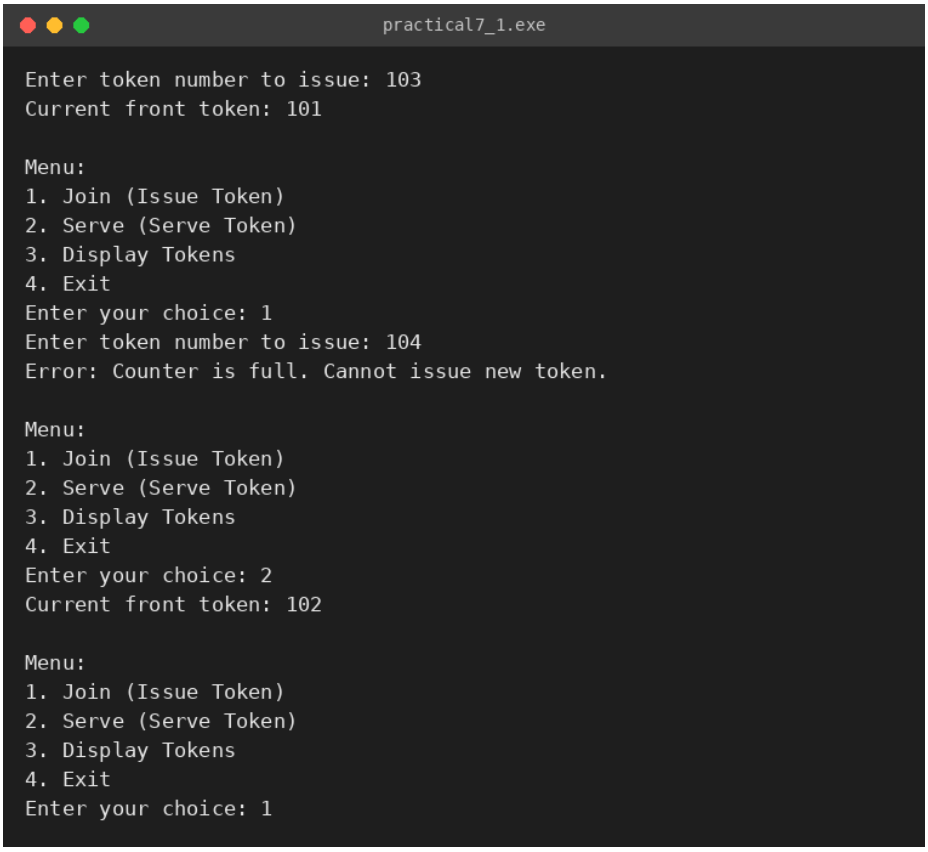
Approach: The queue is a circular array with front, rear and count. A join stores the token at rear = (rear + 1) % capacity and increases count; a serve moves front = (front + 1) % capacity and decreases count. Both operations first check count (full when count == capacity, empty when count == 0) and print an error otherwise. Because the indices wrap around, front and rear keep moving forward over time, but slots freed at the front are reused, so the queue never reports 'full' just because rear reached the last index while free space still exists.

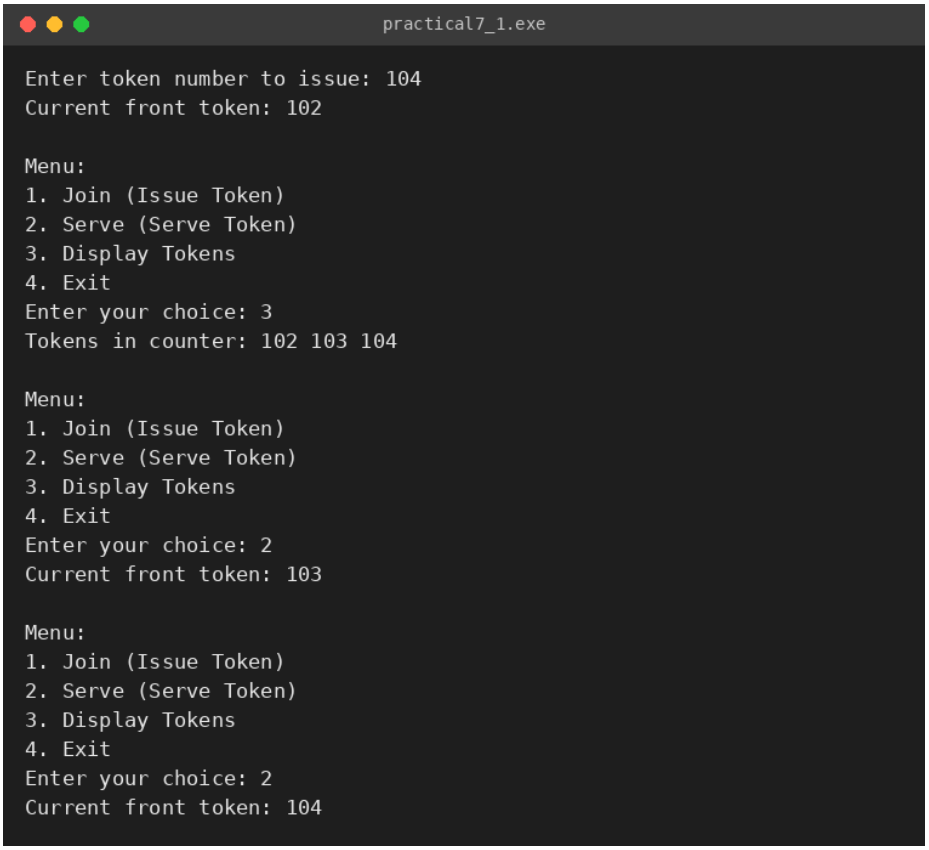
For Code:

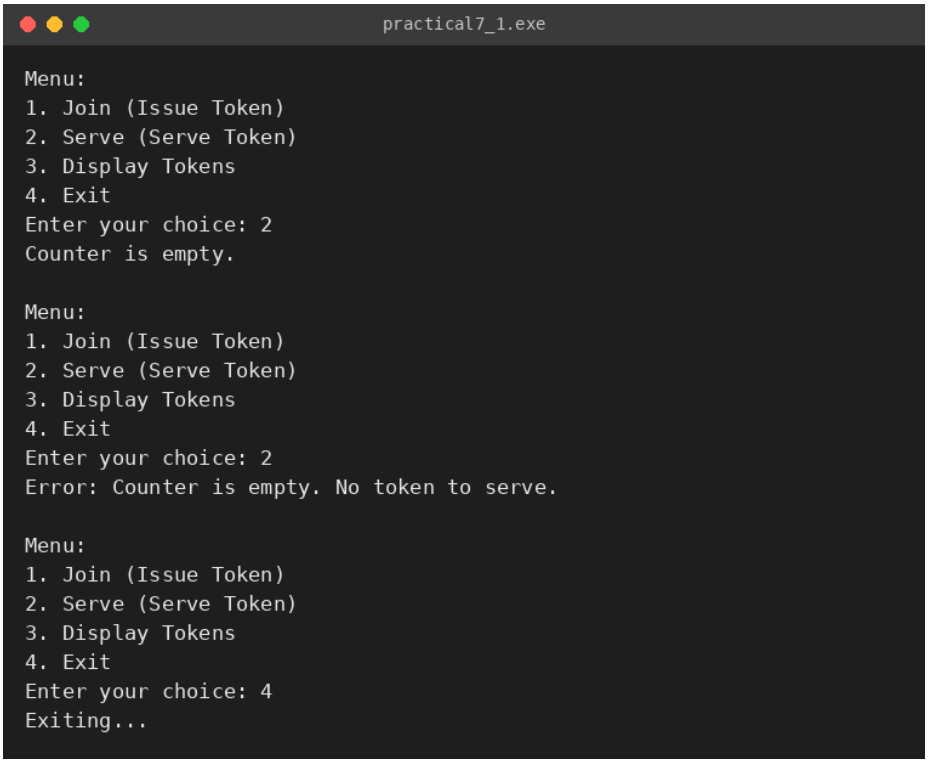
https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-7/practical7_1.cpp

Output :


practical7_1.exe
Enter the maximum number of tokens the counter can hold: 3
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 1
Enter token number to issue: 101
Current front token: 101
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 1
Enter token number to issue: 102
Current front token: 101
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 1


practical7_1.exe
Enter token number to issue: 103
Current front token: 101
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 1
Enter token number to issue: 104
Error: Counter is full. Cannot issue new token.
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 2
Current front token: 102
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 1


practical7_1.exe
Enter token number to issue: 104
Current front token: 102
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 3
Tokens in counter: 102 103 104
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 2
Current front token: 103
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 2
Current front token: 104


practical7_1.exe
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 2
Counter is empty.
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 2
Error: Counter is empty. No token to serve.
Menu:
1. Join (Issue Token)
2. Serve (Serve Token)
3. Display Tokens
4. Exit
Enter your choice: 4
Exiting...

Problem 2:

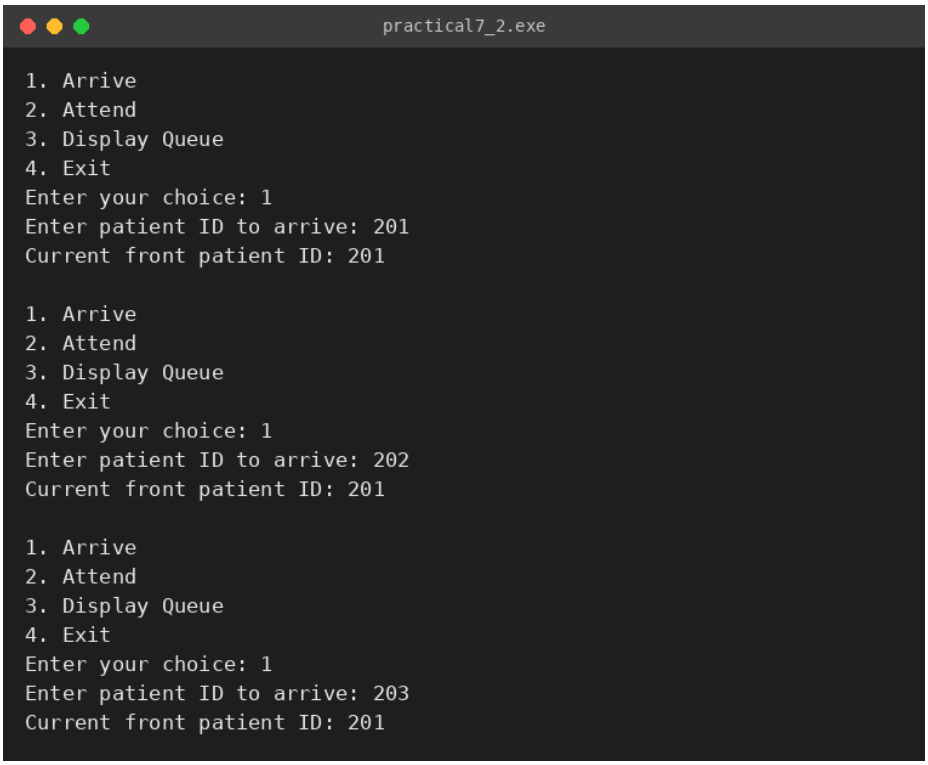
A hospital emergency ward receives patients continuously and attends to them strictly in the order they arrived, with no upper limit on how many can be waiting at once. New patients are added at the back and the doctor always attends to the patient at the front. Given a sequence of arrive and attend operations, implement this unlimited patient queue and print the current front patient after each operation.

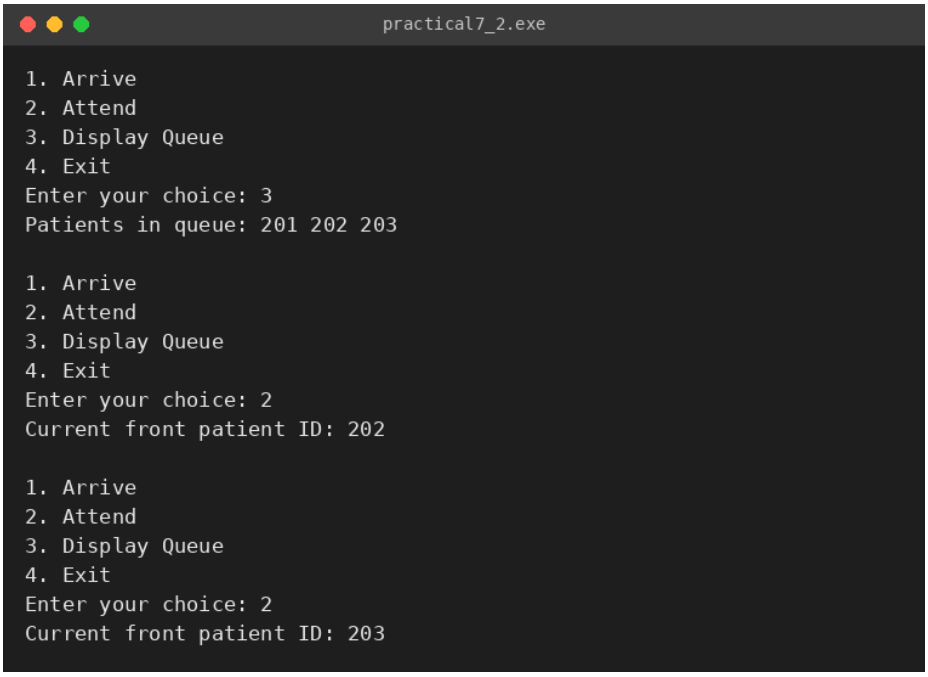
Approach: The queue is a singly linked list with front and rear pointers. Arrive creates a new node at the rear in O(1) and attend removes the front node in O(1). Unlike Problem 1, there is no capacity variable: the queue grows as long as memory is available. If attend is called on an empty ward, the program prints "Error: No patients to attend." instead of dereferencing a null pointer, and when the last patient leaves, rear is reset to null so the queue stays consistent.

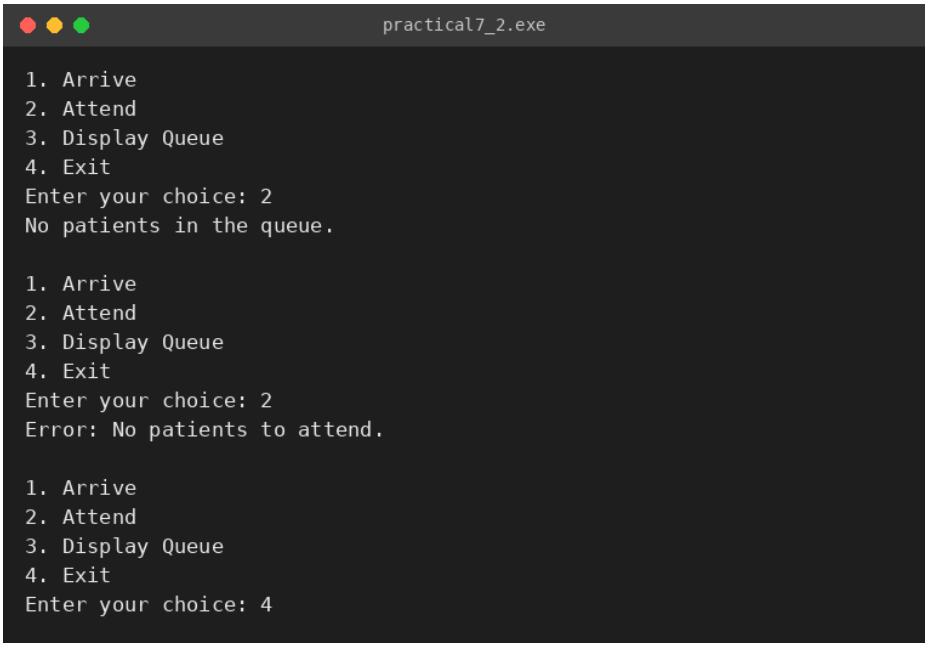
For Code:

https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-7/practical7_2.cpp

Output :







KEY QUESTIONS:

Problem 1: A simple array queue eventually reports being full even when slots have been freed at the front. Explain why this happens structurally — not by tracing an example, but by describing what the front and rear pointers represent and what their movement pattern causes over time.

Problem 2: Your linked list queue has no capacity limit. If arrive operations vastly outnumber attend operations over a long period, what real consequence would this have on the system, and where in your code is that consequence invisible?

ANSWERS:

ANS-1: In a simple array queue, rear always moves forward when someone joins and front always moves forward when someone is served; neither moves backwards. Slots behind front are free, but rear can only advance, so after enough operations rear reaches the last index of the array. The 'full' test (rear == capacity - 1) then becomes true even though many slots at the beginning are empty.

The queue therefore wastes the space already released at the front. Using circular indexing with the modulus operator ((rear + 1) % capacity) lets rear wrap to the freed slots, so the queue is full only when it really holds capacity elements.

ANS-2: If arrivals continuously outnumber attendances, the queue grows without bound. In the real system this means patients wait longer and longer (a growing backlog), and the memory used by the nodes keeps increasing until the system runs out of heap memory, at which point new fails and the program crashes or becomes very slow.

In the code this consequence is invisible because arrive() just calls new Node(...) and links it at the rear; there is no counter, no limit check, and no handling of an allocation failure, so the growth of the queue is never noticed until the memory is exhausted.